

Teoría de Algoritmos

1. Tipos de Datos:

- 1- Numérico
- 2- Alfanumérico
- 3- Lógico o Booleano
- 4- Conjunto o Numerado
- 5- Puntero

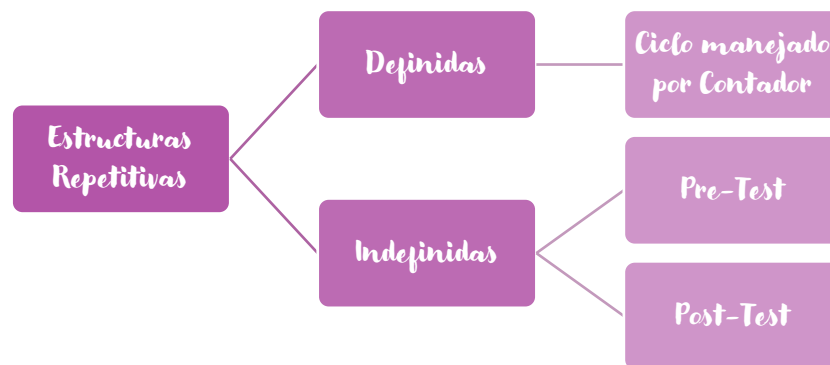
2. Estructuras de Control:

señalan el orden en que tienen que sucederse los pasos de un algoritmo. Se caracteriza por:

- 1- Tener un único punto de entrada(inicio) y un único punto de salida(fin).
- 2- Se compone de otras sentencias o de otras estructuras de control.

Clasificación:

- 1- *Secuenciales*: la ejecución comienza con la primera sentencia del algoritmo y prosigue hasta la ultima sentencia, cada una de las cuales se ejecuta una sola vez.
- 2- *Condicionales*: se realiza una evaluación de una condición y de acuerdo con el resultado, el algoritmo realiza una determinada acción. Pueden ser: Simple, alternativa o múltiple.
- 3- *Repetitivas*:



3- Subacciones:

Acciones que forman parte de una acción principal.

Módulos que están escritos para ejecutar alguna tarea específica. Se definen en el ambiente y son invocadas durante el proceso del algoritmo diseñado. Se escriben una vez, pero pueden ser referenciados en diferentes puntos del algoritmo, de modo que se puede evitar la duplicación innecesaria del código.

Cuando realiza la solicitud, el algoritmo se detiene hasta que el subalgoritmo deja de realizar su tarea, luego continúa; esto se conoce como **control de ejecución**.

Elementos de las Subacciones:

1. Nombre: permite identificarla para su llamada posterior. Debe ser representativo del objetivo de la subacción. Deber único y no deben ocuparse palabras reservadas.
2. Parámetros: de entrada y salida. Son variables y/o constantes para pasar datos entre algoritmos y Subacciones en ambos sentidos.

01 Cantidad de parámetros *El número de parámetros formales debe ser igual al de los argumentos*

02 Orden de los parámetros *Los parámetros que ocupen el mismo orden en cada una de las llamadas deben ser iguales en tipo de dato*

Funciones: es una subacción que recibe datos como argumentos o parámetros, y devuelve un único resultado.

- Funciones internas: las que están incorporadas al sistema.
- Funciones externas: definidas por el usuario.

Procedimientos: no devuelve ningún valor, solo ejecuta acciones.

Ámbito de Variables: lugar donde son declaradas y donde serán utilizadas las variables que forman parte de funciones y procedimientos.

1. Variables Locales: declarada dentro de la subacción (Local a la subacción). Solo está disponible dentro de esta subacción.
2. Variables globales: declaradas en el algoritmo principal, pueden ser utilizados en el algoritmo principal y en todas las Subacciones en el declaradas.

Paso de Parámetros: proceso por el cual el algoritmo principal envía datos a una subacción, de tal forma que estas reciban datos para trabajarlos internamente.

1. **Por Valor:** La información del parámetro se almacena en una dirección de memoria diferente al recibirla en la subacción, por lo tanto si el valor de esa variable cambia no afecta a la variable original, solo se modifica dentro del contexto de la función.
2. **Por Referencia:** La variable que se recibe como parámetro en la subacción apunta exactamente a la misma dirección de memoria que la variable original por lo que si dentro de la función se modifica su valor también se modifica en la variable original.

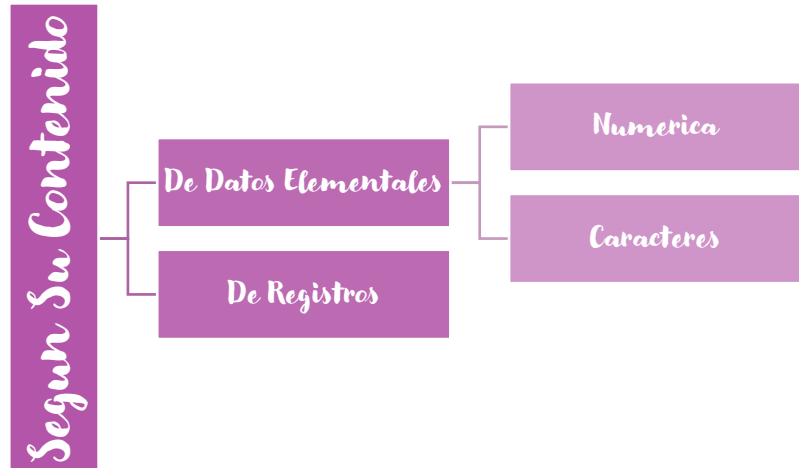
4- Secuencia de Datos Elementales: Conjunto de datos relacionados entre si que se almacena en memoria externa y debe cumplir con ciertas características:

1. **Primer Elemento de la Secuencia:** un objeto del conjunto, llamado primero, se distingue de los demás.
El acceso a este elemento permite el acceso a todos los demás elementos (sucesores) de la secuencia.
2. **Relación de Sucesión:**
 - Todo elemento (excepto el ultimo) precede a otro, o esta antes que otro.
 - Todo elemento (excepto el primer) es el sucesor de otro o sigue a otro.

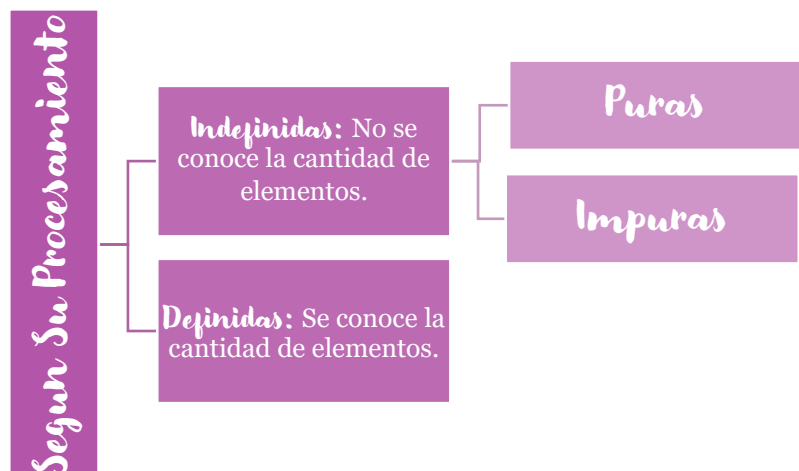
3. **Ultimo Elemento:** Toda secuencia debe poseer un ultimo elemento.
4. **Fin de la Secuencia:** Debe estar definido un indicador de fin de secuencia que puede ser un elemento de la secuencia, una marca, o conocer la cantidad de elementos de la secuencia.

Clasificación:

De acuerdo con el tipo de dato que contienen:



De acuerdo con la cantidad de elementos que contienen:



Secuencias Puras	Secuencias Impuras
Todos los elementos son de la misma especie o tipo. El último elemento es tratado de igual forma que los demás.	Existe un elemento extraño que no debe ser tratado y el cual indica el fin de la secuencia.
Manejada con estructuras post-test.	Manejada con estructura pre-test.

Acciones predefinidas:

1. Arr(sec) : permite iniciar el tratamiento de una secuencia de una secuencia ya existente.
2. Avz(sec. v): permite recuperar el contenido de cada elemento de la secuencia.

3. Crear(sal): permite inicializar una nueva secuencia vacía.
4. Escribir(v): muestra por pantalla el contenido de la variable.
5. Escribir (sec, v): permite guardar datos en una nueva secuencia.

5- Subsecuencias: Conjunto de elementos consecutivos, que están incluidos en la secuencia, y son considerados como un subconjunto de acuerdo con la definición del problema. Son secuencias de menor nivel que pertenecen a una secuencia mayor de la cual heredan sus características.
Esta relación entre subsecuencia y secuencia se puede dar de dos formas:

Subsecuencia Enlazada	Subsecuencia Jerárquica
Están identificadas una a continuación de la otra.	Están identificadas a través de una relación de pertenencia o inclusión.
Se debe definir principio y fin de cada una.	Van desde la de mayor importancia a la de menor importancia.
Son cadenas de Subsecuencias.	Jerarquía de subsecuencias.
Una subsecuencia termina cuando comienza la siguiente.	No hay continuidad ni encadenamiento.

6- Registros: Un tipo de dato “registro” va a estar formado por elementos que pueden ser de diferentes tipos. Es estructura estática, compleja y se almacena en memoria interna.
Heterogeneidad de los datos: diferentes tipos de datos.
Campos de un registro: los elementos que componen un registro pueden ser datos elementales u otras estructuras de datos, a estos elementos se les llama “campo”.
Campo: entidad lógica mas pequeña que conforma al registro. Es la unidad mínima de información de un registro.

- **Elementos de un campo:**
 1. Nombre: Identificador Único del Campo.
 2. Tipo: tipo de dato que puede contener.
 3. Tamaño: que ocupara el campo en el registro.

Campos Contenidos	Campos Continentes
Son campos simples que almacenan un único dato.	Son campos que se encuentran formados por otros mas simples.

Tipos de Campos:

1. Campo clave: identifica al registro de una manera única y lo diferencia de los otros registros.
Clasificación por su formato: pueden ser simples (esta formado por un campo contenido) o complejo (esta formado por un campo continente).
Clasificación por su función:

Primarias	Secundarias	foráneas
Identifica de forma única a un registro.	Sirve para procesar la información en un orden adecuado.	Indica relación con otro archivo.

7- Archivos: es un conjunto homogéneo de registros relacionados entre si y organizados para un propósito específico.
Un archivo esta siempre almacenado en memoria externa y se procesa en la memoria interna.

1.: Organización de Archivos: manera en la que se encuentran almacenados los registros en el archivo.

→ Secuencial: sucesión de registros almacenados consecutivamente (continuidad física).

Para acceder a un registro n es obligatorio pasar por todos los n-1 registros que le pertenecen.

→ Relativa: el orden físico no se corresponde necesariamente con el orden lógico. Se accede a los datos mediante su posición, es decir, el lugar relativo que ocupan.

→ Indexada: Cada archivo indexado consta de un archivo índice y de un archivo de datos.

- Área de datos: contiene los registros en forma secuencial y esta organizada la secuencia de claves sin dejar huecos.
- área de índices: una tabla que contiene los niveles de índice.

Reg 1
Reg 2
...
Reg n-1
Reg n

2.: Acceso de Archivos: Indica la manera en que se recuperan (o leen) los registros en el archivo.

1. Secuencial
2. Directo
3. Mixto

Tratamiento de Archivos:

Abrir: para archivos secuenciales solo se puede abrir en un solo modo (E o S).

- Entrada: archivo de solo lectura.
- Salida: archivos de solo escritura.

Cerrar: liberar recursos utilizados en memoria interna.

Leer: acceder al contenido de sus registros.

No FDA: Controlar el fin del archivo.

8- Procesos con Archivos:

Consistencia: propiedad que verifica la validez del dato almacenado con su definición en el ambiente.

Consistencia automática: La validez se debe comprobar en datos definidos como rango o conjunto. Es decir, datos que tienen limites en su definición en el ambiente.

Congruencia: propiedad que verifica la validez de los datos entre si.

- Fina: validación entre datos en archivos distintos.
- Gruesa: validación entre datos de un mismo registro.

Para que un archivo sea correcto debe cumplir 2 condiciones:

1. Ser consistente: Que la información que esté guardando coincida con el formato usado en la definición.

2. Ser congruente: Se verifica la consistencia de cada elemento consigo mismo. Se compara a un elemento algún otro.

“Si es consistente, no indica que sea congruente”

“Si no es consistente, tampoco es congruente”

Proceso: serie de pasos estructurados y bien definidos que se llevan a cabo para completar un proyecto de manera eficiente y efectiva.

1. Individuales: genérico, emisión, Corte de Control y Procesos Estadísticos.
Un proceso es individual cuando existe un único fichero de entrada y 1 o ningún fichero de salida.

Proceso	Características
genérico o carga: crear un archivo a partir de otro.	Proceso de carga o generación. Crea un archivo consistente.
Emisión: listar información de registros de un archivo dada ciertas condiciones.	Salida Impresa de Datos. Son listadores cuando emiten listados. Se considera padrón cuando los datos ingresados están ordenados y se emiten además totales finales. Padrón: listado ordenado por algún campo.
Corte de Control	Son padrones que poseen totales parciales. Requisito: que el archivo de entrada este ordenado por clave compleja.
Estadísticos	Recorrido de un archivo para contabilizar elementos utilizando una tabla (memoria interna) y al finalizar emitir un cuadro resumen.

2. múltiple: mezcla, actualización.

9- Corte de Control: es una forma ordenada de mostrar información en forma jerárquica.

El corte de control además de totales generales puede emitir también totales de parciales. (Principal Diferencia con el Padrón)

Requisitos para aplicar Corte de Control:

1. Registro debe estar identificado por clave Compleja
2. Los datos se presentan ordenados con respecto al campo o los campos que producen el corte ya que al ordenar, los datos quedan agrupados según se requiera.

10- Mezcla: Proceso en el cual intervienen al menos 2 archivos de entrada, que deben ser combinados para obtener como mínimo uno de salida. Existe una clave que sirve como conector entre los archivos de entrada. Esta mezcla puede ser directa o indirecta.

Qué características tiene el proceso de **MEZCLA**

	DIRECTA	INDIRECTA
Formato de los registros de los Archivos de Entrada	Igual	Distinto
Formato de los registros del Archivos de Salida	Igual al de los Archivos de Entrada	Igual a alguno de los Archivos de Entrada o una combinación
Cantidad de registros del Archivos de Salida	Es igual a la sumatoria de las cantidades de los registros de los archivos de entrada.	No se puede predecir



11- Actualización: el proceso de actualización de un archivo representa

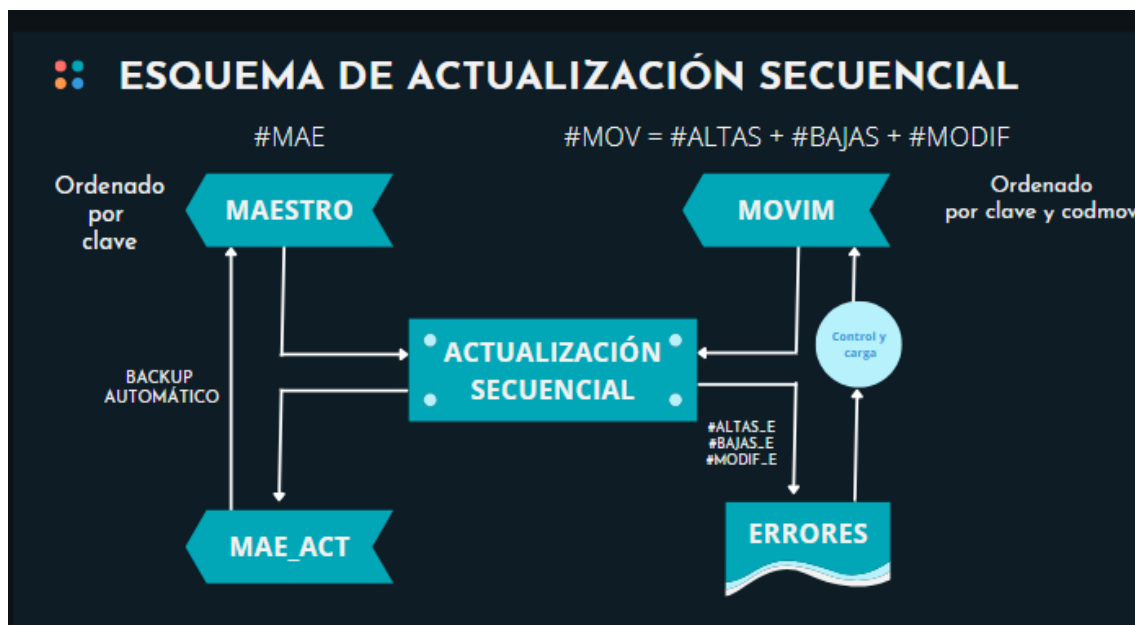
cambios en su organización debido a: un nuevo registro, una modificación o baja de un registro existente.

Elementos de una actualización:

1. Archivo Maestro: representa el archivo que cuenta con la información estática en un momento dado.
2. Archivo de Movimiento: contiene los registros con las novedades para actualizar.

Las novedades que se pueden procesar:

1. Alta: significa que un nuevo registro se añade al archivo.
2. Baja: un registro existente se eliminará del archivo maestro.
3. Modificación: significa que un registro existente se modificara en su contenido.



Actualización Unitaria: Por cada registro del maestro, puede haber 0 o 1 movimiento.

Actualización Por Lotes: varios registros de movimiento para un registro maestro.

12- Actualización Indexada: Se puede acceder al archivo de entrada y de

salida. “Abrir E/S”

Entrada: para obtener información de los registros.

Salida: se utiliza para modificar datos.

Para acceder a los registros: acceso directo o puntual.

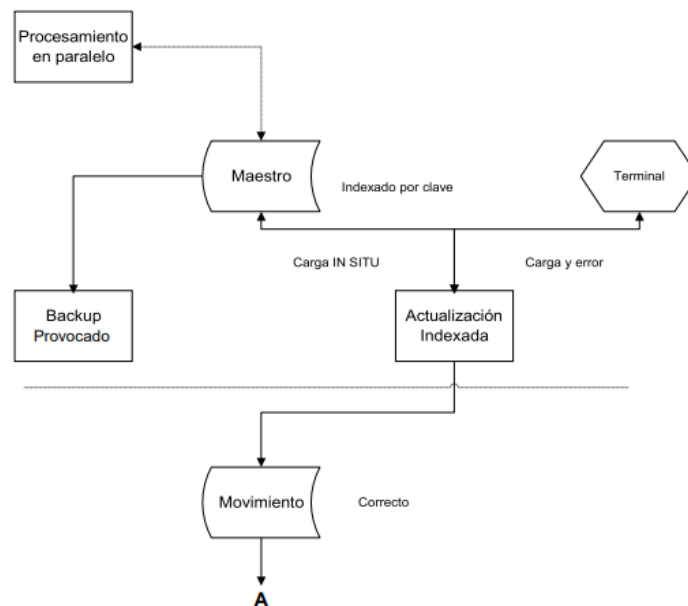
Características:

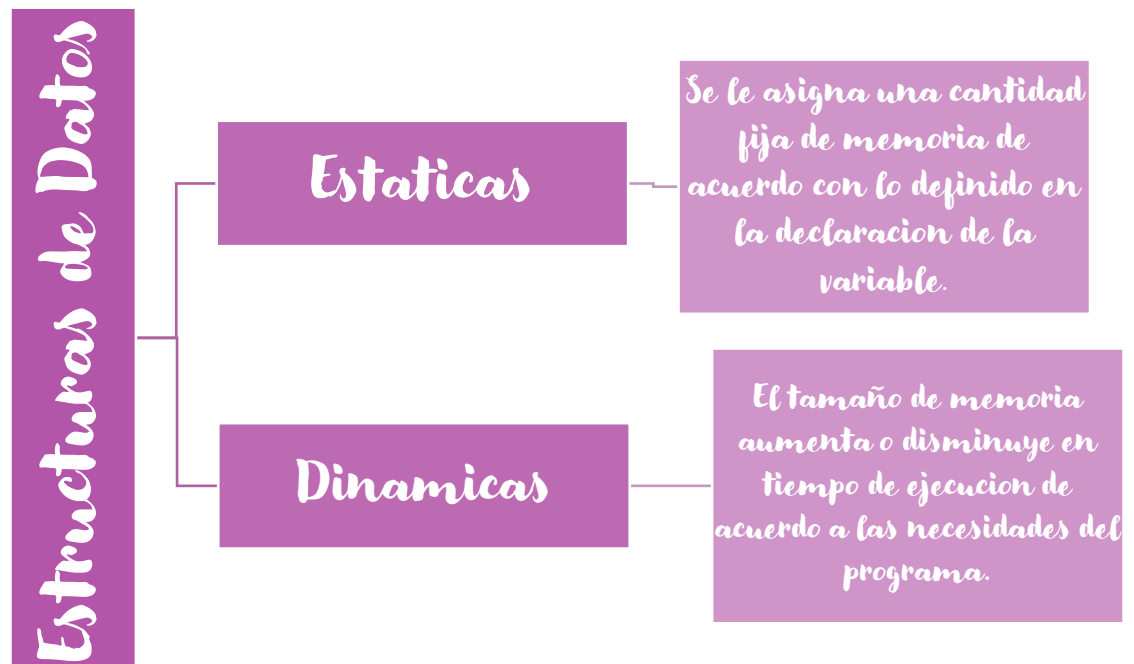
1. Interactivo: cargar los datos y corregirlos durante la carga.
2. In Situ: las modificaciones se realizan directamente sobre el archivo original en su ubicación actual.

3. Inseguro: hay que tomar medidas de control de concurrencia y consistencia.
4. Rápido: Los índices permiten acceder directamente a los registros específicos que necesitan ser modificados, en lugar de tener que buscar secuencialmente a través de todo el archivo.

Actualización Secuencial	Actualización Indexada
Segura	Insegura
Lenta	Rápida
Diferida	In Situ
Batch	Interactiva
Back up automático	Back up intencional

Esquema de Actualización Indexada





14- Arreglos: Es una estructura de datos estática, que es capaz de almacenar, en memoria interna, una colección de datos del mismo tipo. Conjunto homogéneo, finito y ordenado de elementos.

1. **Tamaño:** número de elementos que lo conforman.
2. **Dimensión:** indica la cantidad de índices que necesitamos para acceder a un elemento del arreglo.
3. **Índice:** número de orden de cada elemento dentro del arreglo.

Acciones con arreglos:

1. Lectura/Escritura: acceder a un elemento o asignar un valor a un elemento.
2. Recorrido: acceder a cada uno de los elementos del arreglo para mostrar su valor u operar sobre el.
3. Búsqueda
4. Ordenamiento
5. actualización: modificar el contenido de algún elemento o borrarlo.

Vectores Paralelos: dos o mas arreglos que utilizan el mismo subíndice para acceder a elementos de distintos arreglos. Se pueden procesar simultáneamente.

Recorrido en un arreglo:

1. Recorrido Normal: aquel que permite posicionarse en cada elemento de la estructura respetando su orden y definición.

15- Búsqueda y Ordenamiento (arreglos):

1. **Búsqueda:** involucra recorrer un arreglo con el fin de encontrar un determinado valor.

1- Búsqueda Lineal: compara el elemento que se busca con cada elemento del arreglo. Si son iguales se realiza la acción que determine la consigna y se continua con el recorrido hasta alcanzar la posición N del arreglo. Este proceso encuentra a todos y en cada ocurrencia del elemento buscado con los valores del arreglo, el proceso me permite actuar. El proceso no finaliza

hasta terminar el recorrido del arreglo. Es el único método para búsquedas múltiples. La velocidad de ejecución depende linealmente del tamaño del arreglo.

2- Búsqueda Lineal con Centinela:

- En desordenados: se hace un recorrido manual del arreglo y en el momento de hallar lo buscado, terminar con el recorrido. Como finaliza al encontrar lo buscado, no permite hallar elementos repetidos. Dado que no necesita recorrer totalmente el arreglo es mas veloz que el lineal.
- En ordenados: se hace el recorrido manual del arreglo y en el momento de hallar lo buscado, o en su defecto el inmediato mayor o menor, termina con el recorrido. No permite hallar repetidos. Al estar ordenado es más rápido que para desordenados.

3- Búsqueda Binaria: se aplica en arreglos ordenados. Compara si el valor buscado esta en la mitad superior o inferior. En la que esté, subdivido nuevamente, y así hasta encontrar el valor. Mas grande es el vector y mejor será la performance de este método. Si es pequeño, conviene lineal con centinela para ordenados. En el mejor de los casos el elemento buscado esta en el centro y en el peor de los casos estará en los extremos.

2. Ordenamiento: mover los datos para que queden en una secuencia tal que represente un orden.

1- Selección: El método de ordenamiento por selección consiste en encontrar el menor de todos los elementos del arreglo e intercambiarlo con el que esta en la primera posición. Luego, el segundo más pequeño y así sucesivamente hasta ordenar todo el arreglo.

2- Inserción: Toma uno por uno los elementos de un arreglo y los recorre hacia su posición con respecto a los anteriormente ordenados.

3- Intercambio: La principal característica de este método es el intercambio entre pares de ítems. Se basa en el principio de comparar e intercambiar pares de ítems adyacentes hasta que todos estén ordenados. Realiza solo la cantidad de pasadas necesarias + una extra para verificar que haya quedado ordenado.

16- Estadísticos: Conjunto de resultados cuantitativos que se obtienen de un procesos sistemático de recopilación, tratamiento y divulgación de datos primarios obtenidos sobre hechos que son relevantes para el estudio de fenómenos demográficos, sociales y económicos.

Cuadro Estadístico: arreglo de filas y columnas dispuestas que se puedan presentar y organizar los datos para clasificarlos adecuadamente para comparar e interpretar las características de dos o mas variables.

17-Listas

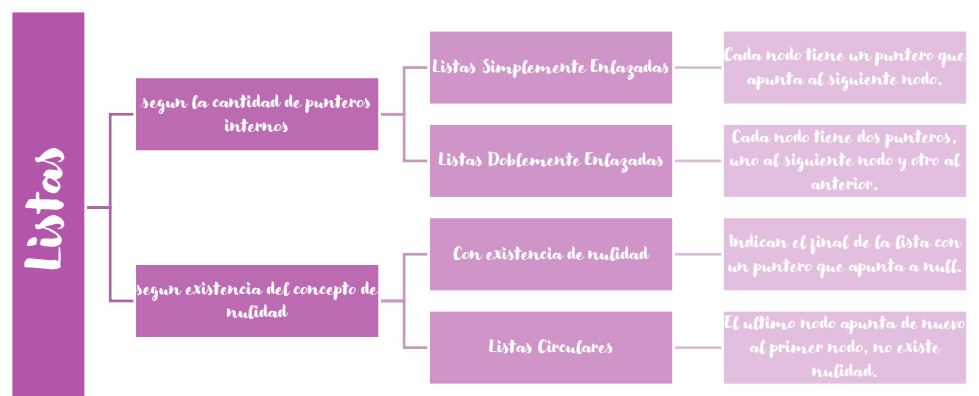
Estructuras Dinámicas: Son aquellas que permiten gestionar la memoria de manera flexible durante la ejecución del programa, adaptándose al tamaño necesario en tiempo real. Esto es útil cuando no se conoce de antemano la cantidad de datos que se van a almacenar.

- Lista Lineal: estructura de datos dinámica donde los elementos, llamados nodos, se organizan secuencialmente y están conectados entre sí mediante punteros. Cada nodo contiene dos partes:
 1. Dato; el valor almacenado.
 2. Enlace: un puntero o enlace que señala al siguiente nodo en la lista.

Las listas lineales se clasifican de acuerdo a la cantidad de punteros tanto externos como internos y a la existencia o no del concepto de nulidad.

Puntero: es una variable que almacena la dirección de memoria de otro elemento almacenar directamente un valor.

- Puntero externo: se encuentran fuera de la estructura de datos y permiten acceder a ella desde el exterior, como el puntero que apunta al primer nodo.
- Puntero interno: internos a la estructura, como enlaces entre nodos en una lista enlazada, conectan un nodo con otros nodos de la lista, permitiendo el recorrido y las operaciones internas.



Acciones:

- Crear nodo
- Agregar elemento
- Buscar elemento
- Borrar elemento
- Recorrer la lista

- Borrar toda la lista
1. **Listas Simplemente Enlazadas:** necesita que cada elemento contenga un enlace con el siguiente elemento.
 2. **Listas Doblemente Enlazadas:** necesita que cada elemento contenga un enlace con el siguiente elemento y un enlace con el anterior elemento. Este tipo de lista permite recorrerse en ambas direcciones, hacia adelante y hacia atrás.
 3. **Listas Circulares:** es una lista lineal en la que el ultimo nodo apunta al primero.

18- Recursividad: es una técnica matemático – computacional en la cual la solución se basa en que el algoritmo realiza llamadas a si mismo y dentro del proceso.

- **Caso Base:** problema trivial que se puede resolver sin calculo. Es una solución simple para un caso particular.
- **Caso Recursivo:** involucra volver a utilizar la función original, con parámetros que se acercan mas al caso base.

Tipos de Recursividad:

1. Directa: se llama a si misma directamente
2. Indirecta: se llaman entre ellas.

Según el modo en que se hace la llamada recursiva la recursividad puede ser:

- De cabeza La llamada se hace al principio del subprograma.
- De cola: la llamada se hace al final del subprograma.
- Intermedia: las instrucciones aparecen tanto antes como después de las llamadas.
- múltiple: se producen varias llamadas recursivas en distintos puntos del subprograma.
- Anidada: Se produce en un parámetro dentro de la propia llamada recursiva.

Por que usar algoritmos recursivos	Cuando no usar algoritmos recursivos
Son mas cercanos a la descripción matemática de problemas	Las soluciones iterativas son usualmente mas rápidas.
Son más fáciles de analizar	Si la memoria de pila es un limitante.
Algunos problemas resultan inabordables iterativamente	Solo se debe usar recursividad cuando no exista una solución iterativa simple.
Legibilidad.	

Diferencias Entre Algoritmos Iterativos y Algoritmos Recursivos:

	Repetición	Terminación
Iteración	Ciclo explicito.	El ciclo termina cuando se llega a la condición de fin.
Recursión	Repetidas invocaciones al proceso recursivo.	Se llega al caso base.

19- Arboles: estructura de datos jerárquica homogénea, no lineal y dinámica. Consiste en nodos conectados por aristas.

- **Homogénea:** todos los elementos son del mismo tipo.
- **Dinámica:** porque tiene la posibilidad de cambiar su forma y cantidad de nodos en tiempos de ejecución.
- **No Lineal:** cada elemento puede tener mas de un sucesor.

Los árboles tienen una definición recursiva.

Beneficios de utilizar arboles:

- Eficacia en Búsqueda
- Estructuración de Datos jerárquicos
- Almacenamiento de Datos Ordenados
- Aplicaciones en computación.

Recorrido en un Árbol Binario: proceso que permite acceder de una sola vez a cada uno de los nodos del árbol. Es examinar completamente a todos los nodos.

Definición en el ambiente :

T= registro

Dato: carácter

Izq, der: puntero a T

Fin Registro

En Orden(IRD):

Subaccion enorden (p: puntero a T) es;

Si (p<>nil) entonces

enorden(*p.izq);

(tratar raíz)

Enorden(*p.der);

Fin si

Fin Subaccion

Pre Orden (RID):

Subaccion preorden (p: puntero a T) es

Si (p<> nil) entonces

(tratar raíz)

Preorden(*p.izq)

Preorden(*p.der)

Fin si

Fin Subaccion

POST-ORDEN (IDR):

Subaccion postorden(p:puntero a T) es

Si ($p \neq \text{nil}$) entonces

Postorden(*p.izq)

Preorden(*p.der)

(tratar raíz)

Fin si

Fin Subacción

- ➔ **Árbol de Expresión:** son aquellos arboles binarios donde las hojas son operandos y los nodos contienen operadores.
- ➔ **Árbol binario equilibrado:** $\text{altura}(\text{subárbol izquierdo}) - \text{altura}(\text{subárbol derecho}) \leq 1$.
- ➔ **Árbol Binario Completo:** aquel en el que todos los nodos tienen dos hijos y todas las hojas están en el mismo nivel. Cada nodo, excepto las hojas, tiene el máximo de hijos que puede tener.
- ➔ **Árbol Binario Distinto:** se dice que dos arboles binarios son distintos cuando sus estructuras son diferentes.
- ➔ **Árbol binario similares:** dos arboles binarios son similares cuando sus estructuras son idénticas, pero la información que contienen sus nodos es diferente.
- ➔ **Árbol binario equivalente:** son aquellos que son similares y además los nodos contienen la misma información.
- ➔ **Árbol ABB (binario de búsqueda):** el hijo izquierdo, si existe, contiene un valor menor que el nodo padre, y el hijo derecho, si existe, contiene un valor mayor que el del nodo padre.
- ➔ **Árbol Balanceado:** es un ABB en el cual se cumple la condición: para todo nodo T del árbol, la altura de los subárboles derechos e izquierdos no debe diferir en más de una unidad.
Factor de equilibrio: la altura de su subárbol izquierdo menos la altura de su subárbol derecho. En un árbol **AVL** el factor de equilibrio es -1, 0 o 1.

20- Complejidad:

Eficiencia de un algoritmo: se refiere a la medida en que utiliza los recursos disponibles de manera óptima para resolver un problema dado. Un algoritmo será más eficiente comparado con otro, siempre que consuma menos recursos, como tiempo y espacio de memoria necesarios para ejecutarlo.

- **Simplicidad:** claridad y facilidad de comprensión del algoritmo. Facilita su verificación, el estudio de su eficacia y su mantenimiento.
- **Recursos:** recursos computacionales necesarios para ejecutar un algoritmo.

1. Complejidad Espacial: la cantidad de espacio en memoria que requiere.
 - Memoria estática: se suma la memoria que ocupan las variables declaradas en el algoritmo.
 - Memoria Dinámica: depende de cada ejecución del algoritmo.
2. Complejidad Temporal: tiempo de computo necesario para ejecutar algún programa.
El tiempo de ejecución del programa va a depender de diversos factores:
 - Los datos de entrada que le suministramos.
 - La naturaleza y rapidez de las instrucciones maquina del procesador concreto que ejecute el programa.
 - La calidad del código.
 - Complejidad intrínseca del algoritmo.

Hay dos estudios posibles sobre el tiempo:

- A priori (medida teórica): obtener una función que acote el tiempo de ejecución del algoritmo para unos valores de entrada dados.
- A posteriori (medida real): Consiste en medir el tiempo de ejecución del algoritmo para unos valores de entrada dados y en una computadora en concreto.

El tiempo de ejecución $T(n)$ de un algoritmo se mide en función a la cantidad de instrucciones que ejecuta.

- $T(n)$ tiempo empleado para ejecutar el algoritmo con un tamaño de entrada n .
- Tamaño de entrada (n): hace referencia al numero de componentes sobre los que se va a ejecutar el algoritmo.

Principio de Invarianza:

Dado un algoritmo y dos implementaciones tuyas I_1 e I_2 , que tardan $T_1(n)$ y $T_2(n)$ respectivamente, el Principio de Invarianza afirma que existe una constante real $c > 0$ y un número natural n_0 tales que para todo $n \geq n_0$ se verifica que $T_1(n) \leq cT_2(n)$.

Para medir el tiempo lo haremos en función del numero de operaciones elementales (OE) que realice dicho algoritmo.

OE: aquellas que una computadora ejecuta en un tiempo acotado por una constante.

El tiempo de ejecución de una secuencia consecutiva de instrucciones se calcula sumando los tiempos de ejecución de cada una de las instrucciones.

El análisis del rendimiento de un algoritmo busca eficientizar su simplicidad y el uso de los recursos disponibles.

El calculo de la complejidad de los algoritmos nos ayuda a conocer su comportamiento asintótico.

Órdenes de complejidad

1. **$O(1)$ Complejidad constante.**- Es la más deseada. Por ejemplo, es la complejidad que se presenta en secuencias de instrucciones sin repeticiones o ciclos
2. **$O(\log n)$ Complejidad logarítmica.**- Por ejemplo la búsqueda binaria.
3. **$O(n)$ Complejidad lineal.**-Por ejemplo el algoritmo de búsqueda secuencial.
4. **$O(n \log n)$ Complejidad cuasi-lineal.**- Por ejemplo algunos algoritmos de ordenamiento por ej. quick sort
5. **$O(n^2)$ Complejidad cuadrática.**- Aparece en ciclos anidados, por ejemplo ordenamiento por selección.
6. **$O(n^3)$ Complejidad cúbica.**- En ciclos triplemente anidados por ejemplo.
7. **$O(n^k)$ Complejidad polinomial.**- Para $(k \in \mathbb{N}, k > 3)$ mientras más crece k , el algoritmo se vuelve más lento
8. **$O(c^n)$ Complejidad exponencial.**- Son programas con funciones que crecen exponencialmente, es decir en cada iteración la cantidad de operaciones se multiplica.
9. **$O(n!)$ Complejidad factorial.**- Esta es una de las peores complejidades que existen, ya que la cantidad de operaciones crece demasiado rápido. Un ejemplo de algoritmo con esta complejidad es aquel que busca todas las permutaciones de un conjunto de números.